Kubernetes Commands

A quick command reference followed by a detailed explanation of how the commands work and when to use them.

Part 1: Command reference

Cluster and context

```
kubectl version                       # client and server versions
kubectl cluster-info                  # control plane endpoints
kubectl get nodes -o wide             # list nodes
kubectl config get-contexts           # list contexts
kubectl config use-context my-cluster  # switch cluster
kubectl config set-context --current --namespace=dev   # default namespace
```

Viewing resources

```
kubectl get pods                      # list pods
kubectl get pods -A                   # all namespaces
kubectl get pods -o wide              # with node and IP
kubectl get pods -l app=web           # filter by label
kubectl get pods -w                   # watch live
kubectl get deploy,svc,ing            # several types at once
kubectl get all                       # common resources in a namespace
kubectl get pod web -o yaml           # full object as YAML
kubectl api-resources                 # all resource types
kubectl explain pod.spec.containers   # built-in field docs
```

Creating and updating

```
kubectl apply -f app.yaml             # create or update from file
kubectl apply -f ./manifests/         # a whole folder
kubectl apply -k ./overlay            # Kustomize
kubectl create deployment web --image=nginx
kubectl create namespace dev
kubectl run test --image=busybox -it --rm -- sh   # temporary pod
kubectl edit deploy web               # edit live object
kubectl patch deploy web -p '{"spec":{"replicas":5}}'
kubectl replace --force -f app.yaml   # delete and recreate
```

Deleting

```
kubectl delete -f app.yaml
kubectl delete pod web-abc12
kubectl delete pods -l app=web
kubectl delete ns dev                 # deletes everything inside it
kubectl delete pod web-abc12 --grace-period=0 --force   # last resort
```

Debugging

```
kubectl describe pod web-abc12        # events, status, reasons
kubectl logs web-abc12                # container logs
kubectl logs -f web-abc12 -c nginx    # follow a specific container
kubectl logs web-abc12 --previous     # logs from the crashed container
kubectl exec -it web-abc12 -- sh      # shell into a pod
kubectl cp web-abc12:/app/log.txt ./log.txt
kubectl port-forward svc/web-svc 8080:80   # local access
kubectl debug -it web-abc12 --image=busybox --target=nginx   # ephemeral container
kubectl get events --sort-by=.lastTimestamp
kubectl top pods                      # CPU and memory (needs metrics-server)
kubectl top nodes
```

Deployments and scaling

```
kubectl scale deploy web --replicas=5
kubectl autoscale deploy web --min=2 --max=10 --cpu-percent=70
kubectl set image deploy/web nginx=nginx:1.28
kubectl rollout status deploy/web
kubectl rollout history deploy/web
kubectl rollout undo deploy/web
kubectl rollout restart deploy/web
```

Services and networking

```
kubectl get svc
kubectl expose deploy web --port=80 --type=ClusterIP
kubectl get endpointslices
kubectl get ingress
kubectl get networkpolicy
```

Config and secrets

```
kubectl create configmap app-cfg --from-literal=MODE=prod
kubectl create configmap app-cfg --from-file=app.properties
kubectl create secret generic db-pass --from-literal=password=s3cret
kubectl get secret db-pass -o jsonpath='{.data.password}' | base64 -d
```

Storage

```
kubectl get pv,pvc
kubectl get storageclass
kubectl describe pvc data-claim
```

Nodes and scheduling

```
kubectl cordon node-a                  # mark unschedulable
kubectl drain node-a --ignore-daemonsets --delete-emptydir-data
kubectl uncordon node-a
kubectl taint nodes node-a key=value:NoSchedule
kubectl label nodes node-a disk=ssd
kubectl label pod web-abc12 tier=frontend
kubectl annotate pod web-abc12 note="test"
```

Security (RBAC)

```
kubectl auth can-i create pods
kubectl auth can-i delete pods --as=jane -n dev
kubectl get role,rolebinding -n dev
kubectl get clusterrole,clusterrolebinding
kubectl create serviceaccount app-sa
kubectl create rolebinding dev-view --clusterrole=view --user=jane -n dev
```

Jobs

```
kubectl create job once --image=busybox -- echo hi
kubectl create cronjob nightly --image=busybox --schedule="0 2 * * *" -- echo hi
kubectl get jobs,cronjobs
```

Output and productivity tips

```
kubectl get pods -o json
kubectl get pods -o jsonpath='{.items[*].metadata.name}'
kubectl get pods -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName
kubectl get pods --sort-by=.status.startTime
kubectl apply -f app.yaml --dry-run=client -o yaml   # preview without changing
kubectl diff -f app.yaml               # show what would change
kubectl create deploy web --image=nginx --dry-run=client -o yaml > app.yaml   # generate YAML
alias k=kubectl
source <(kubectl completion bash)      # tab completion
```

Helm (package manager)

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo nginx
helm install my-web bitnami/nginx
helm upgrade my-web bitnami/nginx --set replicaCount=3
helm list
helm rollback my-web 1
helm uninstall my-web
```

Two habits worth building: use `--dry-run=client -o yaml` to generate manifests instead of writing them from scratch, and use `kubectl describe` plus `kubectl get events` first when something is stuck in `Pending` or `CrashLoopBackOff`.

Part 2: Kubernetes commands in detail

Cluster and context

`kubectl version`

Prints the client version and the server (API server) version. Kubernetes supports a skew of one minor version between `kubectl` and the server, so a large gap can cause odd behavior.

`kubectl cluster-info`

Shows the API server URL and core add-ons like CoreDNS. It's a quick check that you're pointed at the right cluster and that it's reachable.

`kubectl get nodes -o wide`

Lists nodes with status, roles, Kubernetes version, internal and external IPs, OS, and container runtime. `NotReady` in the status column means the kubelet isn't reporting healthy.

Contexts

A context is a combination of cluster, user, and namespace stored in `~/.kube/config`.

- `kubectl config get-contexts` lists them, with `*` marking the current one.
- `kubectl config use-context my-cluster` switches clusters. Check this before running destructive commands in the wrong environment.
- `kubectl config set-context --current --namespace=dev` sets a default namespace so you don't need `-n dev` on every command.

Viewing resources

`kubectl get <type>`

The most-used command. It prints a summary table for a resource type.

- `-A` (`--all-namespaces`) spans all namespaces. Without it, you only see the current namespace.
- `-o wide` adds node and pod IP columns.
- `-l app=web` filters by label. Selectors support `-l 'env in (prod,stage)'` and `-l '!debug'`.
- `-w` streams changes live, which is useful while watching a rollout.
- Short names work: `po`, `deploy`, `svc`, `ing`, `ns`, `pv`, `pvc`, `cm`, `sa`.
- `kubectl get deploy,svc,ing` fetches several types in one call.
- `kubectl get all` shows pods, services, deployments, and replicasets, but not everything (ConfigMaps, Secrets, and Ingresses are missing despite the name).

`kubectl get pod web -o yaml`

Shows the full stored object, including `status` and defaults the API server filled in. It's the best way to see what Kubernetes actually has, as opposed to what you wrote.

`kubectl api-resources`

Lists every resource type in the cluster (including CRDs), with short names, API group, and whether it's namespaced.

`kubectl explain pod.spec.containers`

Built-in documentation for any field path. Add `--recursive` to see the whole tree. It saves a lot of trips to the docs.

Creating and updating

`kubectl apply -f app.yaml`

The declarative way. It creates the object if missing, or merges your changes into the live object if it exists. It records the last applied config in an annotation so it can compute a three-way merge.

- `-f ./manifests/` applies every file in a folder (add `-R` to recurse).
- `-k ./overlay` applies a Kustomize directory.
- `--server-side` moves merge logic to the API server and tracks field ownership, which helps when several tools manage the same object.

`kubectl create ...`

The imperative way. It fails if the object already exists.

- `kubectl create deployment web --image=nginx` makes a Deployment with one replica.
- `kubectl create namespace dev` makes a namespace.

`kubectl run test --image=busybox -it --rm -- sh`

Starts a temporary pod and attaches you to it. `--rm` deletes it when you exit. Everything after `--` is the command to run in the container. It's the standard way to test DNS or connectivity from inside the cluster.

`kubectl edit deploy web`

Opens the live object in your `$EDITOR`, then applies it on save. Convenient for quick fixes, but the change isn't in your YAML files, so it will be overwritten by the next `apply` from them.

`kubectl patch deploy web -p '{"spec":{"replicas":5}}'`

Changes specific fields without a full manifest. The default is a strategic merge patch. Use `--type=merge` for a plain JSON merge, or `--type=json` for precise operations like `[{"op":"replace","path":"/spec/replicas","value":5}]`.

`kubectl replace --force -f app.yaml`

Deletes and recreates the object. This causes downtime for workloads, so use it only for fields that can't be changed in place.

Deleting

- `kubectl delete -f app.yaml` deletes everything defined in the file.
- `kubectl delete pod web-abc12` deletes one pod. If a Deployment owns it, a replacement appears immediately, which is a common way to "restart" a pod.
- `kubectl delete pods -l app=web` deletes by label.

- `kubectl delete ns dev` deletes the namespace and everything in it: pods, services, secrets, PVCs. Double-check the name first.
- `--grace-period=0 --force` removes the object from the API immediately without waiting for the kubelet to confirm the container stopped. The container may keep running on the node, and for StatefulSets this can cause two instances of the same pod. Use it only for pods stuck in `Terminating`, as a last resort.

Debugging

This is the section to know best. A good order when something is wrong is `get`, then `describe`, then `logs`, then `exec`.

`kubectl describe pod web-abc12`

Shows configuration, conditions, and an **Events** section at the bottom. Most failures are explained there: `FailedScheduling` (no node fits), `ImagePullBackOff` (bad image name or credentials), `OOMKilled` (memory limit exceeded), or probe failures.

`kubectl logs web-abc12`

Shows container stdout/stderr.

- `-f` follows live and `--tail=100` limits lines.
- `-c nginx` picks a container when the pod has several.
- `--previous` shows logs from the **last crashed instance**, which is essential for `CrashLoopBackOff`, since the current container may have no useful logs yet.
- `-l app=web` reads logs from all matching pods.

`kubectl exec -it web-abc12 -- sh`

Runs a command inside a running container. Use `-c` to pick a container. Many slim images have no shell, which is where `debug` helps.

`kubectl cp web-abc12:/app/log.txt ./log.txt`

Copies files between pod and local machine. It relies on `tar` being present in the container.

`kubectl port-forward svc/web-svc 8080:80`

Forwards local port 8080 to port 80 of the service (or a pod). The tunnel lasts while the command runs. Good for testing without creating an Ingress.

`kubectl debug -it web-abc12 --image=busybox --target=nginx`

Attaches an **ephemeral container** to a running pod. `--target` shares the process namespace with the named container, so you can inspect it with tools the original image lacks. You can also use `kubectl debug node/node-a -it --image=busybox` to get a shell on a node.

`kubectl get events --sort-by=.lastTimestamp`

Cluster events in time order. Events are kept for about an hour by default, so check them soon after a problem.

`kubectl top pods` / `kubectl top nodes`

Live CPU and memory usage. It requires **metrics-server** to be installed.

Common status meanings:

Status	Usual cause
`Pending`	No node fits (resources, taints, affinity) or PVC not bound

Status	Usual cause
`ImagePullBackOff`	Wrong image or tag, or missing registry credentials
`CrashLoopBackOff`	Container keeps exiting. Check logs --previous
`OOMKilled (in describe)`	Memory limit too low
`Running but not Ready`	Readiness probe failing
`Terminating (stuck)`	Finalizers or an unreachable node

Deployments and scaling

- `kubectl scale deploy web --replicas=5` changes the replica count immediately.
- `kubectl autoscale deploy web --min=2 --max=10 --cpu-percent=70` creates a HorizontalPodAutoscaler. It needs metrics-server and CPU **requests** on the containers, because utilization is measured against requests.
- `kubectl set image deploy/web nginx=nginx:1.28` updates one container's image and triggers a rolling update. The format is `container=image`.
- `kubectl rollout status deploy/web` blocks until the rollout finishes or fails, which makes it handy in CI scripts.
- `kubectl rollout history deploy/web` lists revisions. Add `--revision=2` for details.
- `kubectl rollout undo deploy/web` rolls back to the previous revision. Add `--to-revision=N` for a specific one.
- `kubectl rollout restart deploy/web` does a rolling restart of all pods without changing the spec. It is useful after changing a ConfigMap or Secret that the app only reads at startup.

Services and networking

- `kubectl expose deploy web --port=80 --type=ClusterIP` creates a Service for the Deployment, using its pod labels as the selector. Types are `ClusterIP` (internal only), `NodePort`, and `LoadBalancer`.
- `kubectl get endpointslices` shows which pod IPs are behind each Service. If a Service has no endpoints, either the selector doesn't match pod labels or the pods aren't Ready. This is one of the most common networking problems.
- `kubectl get ingress` and `kubectl get networkpolicy` list those objects. Use `describe` to see rules and backends.

Config and secrets

- `kubectl create configmap app-cfg --from-literal=MODE=prod` creates a ConfigMap from key-value pairs. `--from-file=app.properties` loads a file's content (the filename becomes the key) and `--from-env-file` loads a `.env` file.
- `kubectl create secret generic db-pass --from-literal=password=s3cret` creates an Opaque Secret.
- `kubectl get secret db-pass -o jsonpath='{.data.password}' | base64 -d` reads a value back. Secret values are only **base64-encoded, not encrypted**, so anyone with read access can decode them. Use RBAC, and encryption at rest or an external secrets manager for real protection.

Storage

- `kubectl get pv,pvc` lists PersistentVolumes (cluster-level storage) and claims (requests for storage). A PVC in `Pending` usually means no matching PV or StorageClass.
- `kubectl get storageclass` shows available classes. The one marked `(default)` is used when a PVC doesn't name one.
- `kubectl describe pvc data-claim` shows binding status and events explaining why it is stuck.

Nodes and scheduling

- `kubectl cordon node-a` marks the node unschedulable. Existing pods keep running.
- `kubectl drain node-a --ignore-daemonsets --delete-emptydir-data` cordons the node, then evicts its pods so they reschedule elsewhere. The flags are needed because DaemonSet pods can't be rescheduled and `emptyDir` data is lost on eviction. Drain respects PodDisruptionBudgets, so it may wait.
- `kubectl uncordon node-a` makes the node schedulable again after maintenance.
- `kubectl taint nodes node-a key=value:NoSchedule` repels pods that don't tolerate it. The effects are `NoSchedule`, `PreferNoSchedule`, and `NoExecute` (which also evicts existing pods). Remove a taint by adding a trailing dash: `key=value:NoSchedule-`.
- `kubectl label nodes node-a disk=ssd` adds a label that `nodeSelector` or affinity rules can target. Add `--overwrite` to change an existing label.
- `kubectl annotate ...` adds non-identifying metadata that tools can read.

Security (RBAC)

- `kubectl auth can-i create pods` checks whether you can do something. Add `--as=jane -n dev` to test another user in a namespace, which is great for verifying RBAC changes.
- `kubectl get role,rolebinding -n dev` shows namespace-level permissions. `clusterrole` and `clusterrolebinding` are cluster-wide.
- `kubectl create serviceaccount app-sa` creates an identity for pods.
- `kubectl create rolebinding dev-view --clusterrole=view --user=jane -n dev` grants Jane the built-in read-only role, only inside `dev`. Built-in ClusterRoles are `view`, `edit`, `admin`, and `cluster-admin`.

Jobs

- `kubectl create job once --image=busybox -- echo hi` runs a pod to completion.
- `kubectl create cronjob nightly --image=busybox --schedule="0 2 * * *" -- echo hi` schedules one using cron syntax (here, 02:00 daily).
- `kubectl create job manual-run --from=cronjob/nightly` triggers a CronJob immediately, which is useful for testing.

Output and productivity tips

- `-o jsonpath='{.items[*].metadata.name}'` extracts specific fields. `-o custom-columns=...` builds your own table. `--sort-by=.status.startTime` sorts results.
- `--dry-run=client -o yaml` prints the object instead of creating it. It is the fastest way to generate a starting manifest. `--dry-run=server` also runs API validation and admission.
- `kubectl diff -f app.yaml` shows what `apply` would change. Run it before applying to production.
- `alias k=kubectl` plus `source <(kubectl completion bash)` gives tab completion. For zsh, use `kubectl completion zsh`.
- `kubectl get pods -n dev` vs. `-A`: forgetting the namespace is the most common reason a resource "is not there."

Helm

Helm packages Kubernetes manifests into **charts**, and each install is a tracked **release**.

- `helm repo add` registers a chart repository and `helm search repo nginx` searches it.
- `helm install my-web bitnami/nginx` deploys a chart as a release named `my-web`.
- `helm upgrade my-web bitnami/nginx --set replicaCount=3` changes the release. Use `-f values.yaml` for many overrides. `helm upgrade --install` creates the release if it does not exist, which is handy in CI.
- `helm list` shows releases, and `helm history my-web` shows their revisions.
- `helm rollback my-web 1` returns to revision 1.
- `helm uninstall my-web` removes the release and its resources.
- `helm template` renders manifests locally without installing, which is useful for review.

A debugging walkthrough

For a pod stuck in `CrashLoopBackOff`:

```
kubectl get pods                          # confirm status and restart count
kubectl describe pod web-abc12            # check Events, exit code, OOMKilled
kubectl logs web-abc12 --previous         # see why the last run died
kubectl get pod web-abc12 -o yaml         # verify env, image, probes, limits
kubectl exec -it web-abc12 -- sh          # inspect (if it stays up long enough)
```

If you can see the cause in the logs, the fix is usually in the manifest. After editing, `kubectl apply` and `kubectl rollout status` confirm the result.